Ders 6 Genişletilmiş Notu: Metin Ön İşleme, Precision-Recall Değerlendirmesi ve Ters İndeks

Ders 6 Genişletilmiş Notu: Metin Ön İşleme, Precision-Recall Değerlendirmesi ve Ters İndeks

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Konusu
- 2. Önceki Dersin Tekrarı: Bilgi Erişim Modelleri
- 3. Boolean ve Vektör Uzayı Modellerinde Eşleştirme
- 4. İlgililik Geri Bildirimi
- 5. Metin Ön İşleme
- 6. Erişimin Değerlendirilmesi: Precision ve Recall
- 7. Web Aramasında Değerlendirme ve Zorluklar
- 8. Ters İndeks
- 9. Arama Sırasında Ters İndeksin Kullanılması
- 10. Arama Motorlarını Başarılı Kılan Etkenler

1. Dersin Konusu

Ders, Bing Liu'nun *Web Data Mining* kitabının 6. bölümü olan "Information Retrieval and Web Search" bölümünün kaldığı yerden devam etti. Hoca kitabın ilk dört bölümünün işlendiğini, 5. bölümün (Partially Supervised Learning) işlenmediğini ve bu derste 6. bölümü tamamen bitirmeyi planladığını söyledi. Bölümün slaytları hem kitabın sitesinde hem de dersin sayfasında bulunuyor.

Dersin ilk yarısında hoca, önceki dersi Türkçe olarak tekrar etti. İkinci yarısında İngilizceye geçip metin ön işleme (text pre-processing), değerlendirme (evaluation) ve ters indeks (inverted index) konularını anlattı.

2. Önceki Dersin Tekrarı: Bilgi Erişim Modelleri

Önceki derste bir bilgi erişim motorunun (information retrieval engine) yapısı konuşulmuştu. Elde edilen doküman seti üzerinde belirli bir bilgi erişim modeline dayanarak bir doküman indeksi oluşturulur. Kullanıcıların girdiği sorgular (query) bu indeks üzerinde aranır. Sorguya karşılık gelen sonuçlar sıralanmış biçimde kullanıcıya gösterilir.

Bir bilgi erişim modeli üç şeyi tanımlar:

- Kullanıcının girdiği sorgunun iç temsilini (internal representation),
- Dokümanların iç temsilini,
- Dokümanlarla sorgular arasındaki karşılaştırma ölçütünü.

Dokümanlar bu karşılaştırma ölçütüne göre sıralanıp kullanıcıya gösterilir. Modellerin farklı tipleri vardır: Boolean bilgi erişim modeli, vektör uzayı (vector space) bilgi erişim modeli ve istatistiksel dil (statistical language) modeli. Derste bunlardan Boolean ve vektör uzayı modelleri konuşulmuştu.

2.1 Sözlük ve Vektör Temsili

Bir dokümanı ve bir sorguyu karşılaştırabilmek için ikisini de bir iç temsille, yani bir vektörle tanımlamak gerekir. Elimizde bir doküman seti vardır ve her dokümanın içinde bir takım terimler bulunur. Tek bir dokümandan değil, bütün doküman setinden toplu olarak çıkarılan terimlerin kümesine o doküman setinin sözlüğü (vocabulary) denir.

Hoca bu sözlüğü bir uzay olarak gösterdi. Doküman setinden n tane kelime çıkmışsa n boyutlu bir uzaydan bahsedilir. Doküman setindeki her doküman d de bu uzayda $|V|$ boyutlu bir vektör olarak gösterilir.

2.2 Boolean Modelde Temsil

Boolean bilgi erişim modelinde dokümanın vektöründe, dokümanda geçen kelimelerin yerine 1, geçmeyenlerin yerine 0 konur. Sorgu da aynı şekilde tanımlanır. Sorguya anahtar kelimeler (keyword) ya da anahtar kelime grupları girilir. Sözlükteki kelimelerden sorguda bulunanlara 1, bulunmayanlara 0 verilir.

Burada temsil edilen sorgu, çalıştırılabilir sorgudur (executable query). Hoca bunun ne olduğunu tekrar hatırlattı. Arama motoru kullanıcının girdiği sorguyu alır. Gerekirse kendisi de bazı ek anahtar kelimeler ekler. Sonra kelimeler arasında AND, OR, NOT operatörlerini kullanarak sorguyu çalıştırılabilir hâle getirir.

Arama motorları kimi zaman, kullanıcıya göstermeden sorguya kelime ekler:

- Kullanıcının daha önce yaptığı aramalardan çıkardığı kelimeler,
- Kullanıcının bulunduğu konumdan çıkardığı kelimeler.

Hocanın örneği: Bir elektronik cihaz için anahtar kelimeler girildiğinde, arama motoru bunların yanına kullanıcının bulunduğu şehir ve ülkenin adını da ekleyerek arama yapar. Böylece gelen sonuçlarda kullanıcıya yakın elektronik mağazaları çıkar.

2.3 Vektör Uzayı Modelinde Temsil: TF-IDF

Vektör uzayı modelinde vektörün elemanları için TF-IDF denen bir ölçü kullanılır. Bu ölçü iki bölümden oluşur.

İlk bölüm terim frekansıdır (term frequency, TF). Bir kelimenin bir doküman içindeki önemini ifade eder. Kelimenin o dokümanda kaç kez geçtiği, dokümanda en çok tekrar eden kelimenin tekrar sayısına bölünür. Yani normalize edilir:

$$tf_{ij} = \frac{f_{ij}}{\max\{f_{1j}, f_{2j}, \dots, f_{|V|j}\}}$$

Burada f_{ij}, i. terimin j. dokümanda geçme sayısıdır. Bu değer, kelimenin bu doküman için ne kadar önemli olduğunu söyler.

İkinci bölüm ters doküman frekansıdır (inverse document frequency, IDF). Doküman setinde çok sayıda doküman vardır. Bir kelimenin kaç dokümanda geçtiğine doküman frekansı (document frequency, df_i) denir:

$$idf_i = \log \frac{N}{df_i}$$

Burada N, doküman setindeki toplam doküman sayısıdır. Kelime çok sayıda dokümanda geçiyorsa IDF düşük, az sayıda dokümanda geçiyorsa yüksek olur. Hocanın akılda kalması için verdiği kural şudur: Bir kelime bütün dokümanlarda geçiyorsa önem değeri düşüktür. Az sayıda dokümanda geçiyorsa, nadir bulunan bir kelime olduğu için önem değeri büyüktür.

Dokümandaki her kelimenin TF ve IDF değerleri bulunur ve vektördeki ağırlık bu ikisinin çarpımı olur:

$$w_{ij} = tf_{ij} \times idf_i$$

Kelime dokümanda yoksa vektördeki değeri yine 0'dır. Boolean modelden farkı, kelime varsa 1 yerine TF-IDF değerinin konmasıdır.

3. Boolean ve Vektör Uzayı Modellerinde Eşleştirme

İki model, sorgu ile doküman arasındaki eşleştirmede de ayrılır.

Boolean modelde tam eşleşme (exact match) aranır. Kullanıcının sorgusu alınır, iç temsili belirlenir ve operatörlerle birleştirilir. Bu sorguyu true yapan dokümanlar listelenir.

Vektör uzayı modelinde ise doküman ile sorgu arasında bir benzerlik ölçüsü (similarity metric) kullanılarak ne kadar benzer oldukları anlaşılmaya çalışılır. Bunun için kosinüs benzerliği (cosine similarity) kullanılır. Kosinüs benzerliği, iki vektörün iç çarpımının (dot product) vektörlerin uzunluklarının çarpımına bölümüdür:

$$cosine(\mathbf{d}_j, \mathbf{q}) = \frac{\mathbf{d}_j \cdot \mathbf{q}}{\|\mathbf{d}_j\| \times \|\mathbf{q}\|}$$

> **Ders sonrası not:** Hoca "cosine similarity'yi bularak iki vektörün arasındaki uzaklığı buluyorum" dedi. Kosinüs bir uzaklık değil benzerlik ölçüsüdür. Değer büyüdükçe vektörler birbirine daha yakındır. Uzaklık olarak kullanılmak istenirse büyük olasılıkla $1 - cosine$ gibi bir dönüşüm gerekir.

Hoca slaytlardaki örneği hatırlattı. Aynı doküman seti ve sorgu için Boolean model kullanılsaydı ve vektör uzayı modelinde kosinüs benzerliği kullanılsaydı kullanıcıya gösterilecek sonuçların ne olacağı karşılaştırılmıştı.

Kosinüs benzerliğinin alternatifleri de vardır. Bunlardan biri Okapi benzerlik ölçüsüdür. Slaytta Okapi ile bir doküman ve bir sorgu arasındaki benzerliğin formülü gösterilmişti.

> **Hocanın vurgusu:** Okapi formüllerini ezberlemenize gerek yok, kosinüs benzerliği önemlidir. Okapi, doküman ile sorgu arasında başka ölçüler de olduğunu bilmeniz için paylaşıldı.

4. İlgililik Geri Bildirimi

Önceki derste konuşulan bir diğer konu ilgililik geri bildirimiydi (relevance feedback). Hoca burada Google, Bing, Yandex gibi genel arama motorlarını değil, ACM ve IEEE gibi daha özelleşmiş arama motorlarını kastettiğini belirtti.

Bu arama motorlarının bazılarında süreç şöyle işler:

- Kullanıcı bir anahtar kelime ya da anahtar kelime grubuyla arama yapar.
- Arama motoru sonuçları gösterir ve her sonucun yanına bir onay kutusu (checkbox) koyar. Kullanıcıdan ilgili (relevant) sonuçları işaretlemesini ister.
- Kullanıcı ilgili olanları işaretler ve alttaki "Refine Search Results" düğmesine basar.
- Sistem, ilgili diye işaretlenen sayfalarda en yüksek frekansta geçen terimleri çıkarır ve bunları kullanıcının ilk sorgusu q'ya ekler.
- İlgisiz diye bırakılan sayfalardaki kelimeleri de çıkarır. Bu kelimeler ilk sorguda varsa sorgudan atar.
- Böylece genişletilmiş bir sorgu (expanded query) oluşur ve arama bu yeni anahtar kelime grubuyla tekrarlanır.

Hocanın örneğinde kullanıcı sorguya iki anahtar kelime girmişse, sistem bunu beş anahtar kelimeye dönüştürür.

Bu yöntem, çok büyük bir doküman setinde tam olarak aranan şeyi bulmak için kullanışlıdır. Özellikle kullanılan anahtar kelimeler çok sayıda dokümanda geçebilen kelimelerse, arama motoru bu yöntemle aranan şeyin daha hızlı bulunmasını sağlar. Hoca bunun her arama motorunda bulunmadığını, bazı özelleşmiş arama motorlarında görülebildiğini söyledi.

5. Metin Ön İşleme

Hoca İngilizceye geçerek yeni konuya başladı ve bilgi erişim mimarisine geri döndü. Şimdiye kadar şunlar konuşulmuştu: kullanıcının sorguyu nasıl girdiği, sorgunun nasıl çalıştırılabilir sorguya dönüştüğü, sorgunun ve dokümanın içte nasıl tanımlandığı, doküman ile sorgunun nasıl karşılaştırıldığı ve sonuç kümesinin nasıl bulunduğu.

Bu kısımda ise doküman koleksiyonundaki dokümanların nasıl işlendiği anlatıldı. Amaç, doküman indeksi (document index) denen bir tablo oluşturmak ve aramayı bu indeks üzerinde yapmaktır. Bu işi mimarideki indeksleyici (indexer) yapar.

Metin ön işlemede yapılan işler sırasıyla şunlardır:

- Dokümanlardan terimler çıkarılır. Yalnızca anlamı olan terimler çıkarılmak istenir, durak kelimeler (stop words) çıkarılmaz.
- Kök bulma (stemming) uygulanır. Bu, çok daha etkili indeks tabloları sağlar.
- TF-IDF değerleri hesaplanır.

5.1 Durak Kelimeler

Her dilde kendi başına anlamı olmayan bazı kelimeler vardır. Bu kelimeler cümlelerde başka kelimelerle birlikte kullanılır ve cümleyi anlaşılır kılar. Dilin dilbilgisi kurallarına göre cümlelerde yer alırlar, ama kendi başlarına anlam taşımazlar. İngilizcede “the”, “of”, “to” gibi kelimeler bu türdendir. Türkçede de bağlaçlar böyledir. Bağlaçların kendi başlarına anlamı yoktur, ama cümle içinde, isim tamlamalarında ve anahtar kelime gruplarında kullanılırlar.

Bu kelimelerin dokümanlardan çıkarılan terimler arasında yer alması istenmez. Bu yüzden terimler çıkarılırken durak kelimeler atılır. Hocaya göre bu yapıldığında indeksin boyutu yaklaşık %30 küçülür.

5.2 İndeksin Görünümü

Hoca ilerlemeden önce indeksin neye benzediğini gösterdi ki sonraki anlatım daha iyi anlaşılsın. Slayttaki basitleştirilmiş indeks iki sütundan oluşur:

- **Birinci sütun:** Anahtar kelimeler. Bu, bütün dokümanlardan çıkarılan kelimelerdir, yani doküman setinin sözlüğü ya da anahtar kelime uzayıdır.
- **İkinci sütun (değer kısmı):** Bu anahtar kelimeyi içeren dokümanların listesi. Bir anahtar kelime üç farklı dokümanda geçiyorsa burada üç farklı doküman görülür.

Bu oyuncak (toy) örnekte üç doküman vardı ve her dokümanda farklı cümleler bulunuyordu. Oluşturulmak istenen tablo budur. Terimler çıkarılırken durak kelimeler atıldığı için indeksin girdilerinde yalnızca anlamı olan anahtar kelimeler kalır ve indeksin boyutu küçülür.

5.3 Kök Bulma

İkinci adım kök bulmadır (stemming). Hoca bunu Türkçe olarak “kökünü bulmak” diye açıkladı: Kelimenin bütün biçimlerini indekslemek yerine yalnızca kökü bulunur ve kök indekslenir.

Slayttaki örnekler şunlardı:

- Dokümanda “users”, “used”, “using” biçimleri geçiyor. Bu kelime grubuna kök bulma uygulanınca kök “use” olur. Bu kelimelerin hepsini indekslemek için yalnızca bir girdi kullanılır.
- “engineering”, “engineers”, “engineer” kelimeleri de indekslenirken tek bir anahtar kelimeyle temsil edilir.

Hocaya göre kök bulma, toplam indeks boyutunu da %40-50 oranında küçültür.

> **Ders sonrası not:** Hocanın verdiği %30 ve %40-50 oranları kaba tahminlerdir. Gerçek kazanç büyük olasılıkla dile, koleksiyona ve indeks boyutunun neyle ölçüldüğüne (sözlükteki terim sayısı mı, posting sayısı mı) bağlıdır.

Her dilin kendi kök bulma kuralları vardır. Slayttaki örnek İngilizce içindi. Türkçenin de kendi kök bulma kuralları vardır.

> **Ek bilgi (derste anlatılmadı):** İngilizce için en yaygın kök bulma algoritması Porter Stemmer'dır. Kelimenin sonundaki ekleri ("-ing", "-ed", "-s" gibi) sıralı kurallarla kırpar. Türkçe sondan eklemeli bir dil olduğu için kök bulma daha zordur. Bu yüzden Türkçede büyük olasılıkla Zemberek gibi biçimbilimsel çözümleyiciler kullanılır.

Durak kelime çıkarma ve kök bulma, kural tabanlı ayrıştırıcılardır (rule-based parser).

> **Hocanın vurgusu:** İndeks tablosu oluşturulmadan önce, dokümandan anahtar kelimeler çıkarılırken önce durak kelimeler atılır, sonra kök bulma uygulanır. Böylece sonunda dokümanı gerçekten temsil eden anahtar kelimeler elde edilir.

5.4 TF-IDF Hesabı ve Dokümanın Vektörle Temsili

Doküman setindeki her dokümandan anahtar kelimeler çıkarıldıktan sonra bu dokümanları temsil etmek gerekir. Vektör uzayı modelinde bir doküman temsil edilirken ağırlık olarak TF-IDF kullanılıyordu. TF terim frekansı, IDF ters doküman frekansıdır. TF-IDF'in nasıl hesaplandığı daha önce konuşulmuştu.

Verilen bir dokümandaki her anahtar kelime için TF-IDF değeri hesaplanır. Her dokümandaki her anahtar kelimenin TF-IDF değeri hesaplandıktan sonra, bu dokümanlar birer vektör olarak temsil edilebilir.

Hoca mimariye geri dönerek özetledi: Dokümanlara metin ön işleme uygulandı. Bu ayrıştırıcıların hepsi indeksleyicinin parçasıdır. Hepsi uygulanarak indeks tablosu oluşturulur. İndeks tablosunda her doküman bir vektörle temsil edilir. Vektör uzayı modelinde bu vektörün elemanları TF-IDF ile hesaplanır.

6. Erişimin Değerlendirilmesi: Precision ve Recall

Tablo oluşturuldu, sorgular bu tabloya uygulandı, dokümanlar getirildi, sıralandı ve kullanıcıya gösterildi. Peki kullanıcıya gösterilen sonuç kümesinin iyi bir sonuç kümesi olduğunu nereden bileceğiz? Değerlendirme, bir bilgi erişim mimarisinin önemli bir parçasıdır.

Değerlendirme için iki ölçü kullanılır: kesinlik (precision) ve duyarlılık (recall).

6.1 İki Soru

Precision, getirilen dokümanların ilgili olup olmadığıyla ilgilenir. Şu soruyu cevaplar: Getirilen bütün dokümanlar ilgili mi? Örneğin kullanıcıya ilk 10 sonuç gösteriliyorsa, bu 10 sonucun kaç tanesi ilgili?

Recall ise şu soruyu cevaplar: İlgili dokümanların hepsi getirildi mi? Hoca bunu Türkçe olarak şöyle ifade etti: Kullanıcıya gösterilen sonuçlardaki ilgili dokümanlar, toplamdaki ilgili dokümanların ne kadarıdır?

6.2 Kümelerle Tanım

Slaytta bir şekil vardı. Doküman koleksiyonunun içinde iki küme gösteriliyordu:

- **Yeşil elips (B):** Verilen bir sorgu için bilgi erişim sisteminin getirdiği dokümanlar.
- **Mor şekil (A):** Bütün doküman koleksiyonu içinde sorguyla ilgili olan dokümanlar.

Getirilen dokümanların bir kısmı sorguyla ilgisiz, bir kısmı ilgilidir. İlgili ve getirilmiş olanlar $A \cap B$ kesişimidir.

Precision, kullanıcıya gösterilen dokümanlar içindeki ilgili dokümanların sayısının, kullanıcıya gösterilen toplam doküman sayısına bölümüdür:

$$precision = \frac{|A \cap B|}{|B|}$$

Recall, kullanıcıya gösterilen ilgili dokümanların ($A \cap B$) toplamdaki ilgili dokümanlar içindeki oranıdır. Payda A, pay $A \cap B$'dir:

$$recall = \frac{|A \cap B|}{|A|}$$

Ders sonrası not: Hoca recall'u anlatmaya başlarken önce "among the retrieved relevant documents" dedi. Hemen ardından doğru tanımı verdi: Recall, gösterilen ilgili dokümanların toplam ilgili doküman sayısına oranıdır. Payda getirilen dokümanlar değil, bütün ilgili dokümanlardır.

6.3 Sayısal Örnek 1: 15 İlgili Doküman, İlk 10 Sonuç

Hocanın örneği:

- **Verilen:** Doküman setinde, verilen bir sorgu için toplam 15 ilgili doküman var, yani $|A| = 15$. Bilgi erişim sistemi kullanıcıya ilk 10 sonucu gösteriyor, yani $|B| = 10$. Bu 10 sonucun yalnızca 5'i ilgili, yani $|A \cap B| = 5$.
- **Precision (ara adım eklendi):** $precision = \frac{5}{10} = 0{,}5$.
- **Recall (ara adım eklendi):** $recall = \frac{5}{15} = \frac{1}{3} \approx 0{,}33$.

Hoca bu örnekte iki soruyu Türkçe olarak ayırdı: İlk 10 sonuçta 5 ilgili sonuç varsa, birinci soru precision'ı verir. İkinci soru, bu 5 ilgili sonucun toplamdaki 15 ilgili doküman içindeki payıdır, bu da recall'dur.

6.4 Sayısal Örnek 2: İlk 10'da 3 İlgili Sonuç

Hoca ardından Precision@10 (precision top 10) ölçüsünü anlattı:

- **Verilen:** Kullanıcıya ilk 10 sonuç gösteriliyor ve kullanıcı bunlardan 3'ünü ilgili buluyor.
- **Sonuç:** Bu bilgi erişim sisteminin Precision@10 değeri $\frac{3}{10} = 0{,}3$ olur.

Hoca bundan sonra ikinci bir bilgi erişim sistemini aynı yolla değerlendirip ilk sistemle karşılaştırdı.

7. Web Aramasında Değerlendirme ve Zorluklar

7.1 Global Arama Motorlarında Recall Bilinemez

Global arama motorlarında recall değeri bilinemez. Çünkü verilen bir anahtar kelime için toplamda kaç ilgili doküman bulunduğu bilinmez. Ancak üzerinde çalışılan küçük bir doküman koleksiyonu varsa, ilgili doküman sayısı ve dolayısıyla recall bilinebilir.

Precision ölçüsü ise global arama motorlarında yaygın olarak kullanılır. Ama Precision@10, Precision@20, Precision@30 gibi özelleştirilmiş biçimleri kullanılır.

7.2 Arama Motorlarını Karşılaştırmak

Arama motorlarının başarısı şöyle karşılaştırılır:

- Verilen bir sorgu için her arama motorunun ilk 10'da gösterdiği sonuçlar alınır.
- Bir kullanıcı anketi yapılır. Kullanıcılara gösterilen sonuçlardan hangilerini ilgili kabul ettikleri sorulur.
- İnsanların verdiği yanıtlara göre Google, Bing, Yandex, Yahoo gibi motorların Precision@10, Precision@20, Precision@30 değerleri bulunur.
- Bu işlem tek bir anahtar kelime için değil, çok sayıda anahtar kelime için yapılır ve ortalama alınır.

Slaytta farklı sorgular (q_j) için precision değerlerinin bulunduğu ve bu değerlerin ortalamasının alındığı gösteriliyordu.

7.3 Precision-Recall Eğrisi

Arama motorlarında precision ve recall ölçüleri yaygın olarak birlikte kullanılır. Precision-recall eğrisinde x ekseninde recall değerleri, y ekseninde precision değerleri bulunur. Recall için sabit değerler tutulur: %10, %20, %30 diye %100'e kadar gidilir. Her biri için precision değeri hesaplanır.

Hoca bu eğrinin ancak, bir kelimenin doküman seti içinde toplam kaç dokümanda ilgili olarak geçtiğinin bilindiği doküman setleri üzerinde çizilebileceğini tekrar vurguladı.

> **Ek bilgi (derste anlatılmadı):** Liu'nun kitabında bu eğri, sıralanmış sonuç listesinde her ilgili doküman bulunduğunda precision ve recall'un yeniden hesaplanmasıyla çizilir. Sabit recall noktalarındaki precision büyük olasılıkla interpolasyonla bulunur. Birden çok sorgu varsa her recall noktasındaki precision değerlerinin ortalaması alınarak tek bir eğri elde edilir.

7.4 Web Aramasının Zorlukları

Web'de precision ve recall nasıl uygulanır sorusunun cevabı az önce verildi: Precision@10, bazen @20, bazen @30 kullanılır. Hoca, web arama motorlarının işinin küçük ölçekli bilgi erişim motorlarına göre çok daha zor olduğunu söyledi.

Birinci zorluk sorguların geniş olmasıdır. Kullanıcılar genelde geniş konular girer. Girilen kelimelerin birden fazla anlamı olabilir. Bilgi erişim motoru, kullanıcının bu kelimeleri girerken niyetinin ne olduğunu anlamalı ve bu niyete en iyi uyan sonuçları bulmaya çalışmalıdır. Hangi anlamla aranması gerektiğini anlamak zordur. Bu yüzden ilk 10 sonuçta kullanıcının gerçekten istediği bütün sonuçları getirmek gerçekten zordur. Kullanıcı arama motorunun işini kolaylaştıracak anahtar kelime grupları girmiyorsa, hangi arama motoru olursa olsun ilk 10 sonuçta yüksek precision yakalamak zordur.

İkinci zorluk, aramanın çok büyük bir doküman seti üzerinde yapılmasıdır. Verilen anahtar kelimeler çok sayıda dokümanda ilgili olarak geçer. Bu ilgili dokümanlardan hangilerinin getirileceğine karar vermek gerekir.

> **Ders sonrası not:** Hoca bu noktada "bu doküman seti içerisinde recall değeri yüksek olacak" dedi. Kastedilen, ilgili doküman sayısının ($|A|$) çok büyük olmasıdır. Recall'un paydası büyüdüğü için, ilk birkaç sonuçla elde edilen recall büyük olasılıkla tersine çok düşük kalır.

Arama motorları bu zorluklarla şöyle başa çıkar:

- **Sorgu genişletme:** Girilen anahtar kelimeler çoğu zaman genel kelimelerdir ve biraz daha özelleştirilmeleri gerekir. Arama motorları kullanıcının daha önce girdiği sorgu geçmişine ya da bulunduğu konuma bakar. Bunlardan yola çıkarak sorguyu genişletir ve öyle arar.
- **Otoriter sayfalar:** Bulunan ilgili dokümanlardan hangilerinin gösterileceğine karar verirken öncelikle otoriter (authoritative), yani popülerliği daha yüksek sayfaları seçerler. Bunu hesaplarken bir sonraki bölümde görülecek bağlantı analizi (link analysis) yöntemlerini kullanırlar.
- **Çeşitlilik (diversity):** Sonuçların hepsi tek bir alan adından (domain) getiriliyorsa arama motorunun başarısı düşük olur. Çünkü sonuçlar tek bir sitenin bakış açısını yansıtır. Getirilen sonuçların her biri farklı bir siteden gelmelidir.

8. Ters İndeks

8.1 Doküman Koleksiyonu Nereden Gelir?

Hoca mimariyi bir kez daha özetledi: Elimizdeki doküman koleksiyonunun üzerinde indeksleyici modeli kullanılır ve metin ön işleme teknikleri uygulanarak bir indeks tablosu oluşturulur.

Doküman koleksiyonunu ise web tarayıcısı (web crawler) denen bir uygulama oluşturur. Web tarayıcısı web'deki sayfaları bulur, içeriklerini indirir ve doküman koleksiyonuna koyar. Bütün dokümanlar koleksiyonda toplandıktan sonra metin ön işleme teknikleri uygulanır. Tarayıcı konusu ara sınavdan sonra işlenecek.

8.2 İndeks Tablosu ve Ters İndeks

İndeksleyici modelinde metin ön işleme uygulandıktan sonra bir indeks tablosu oluşturulur. İndeks tablosu, arama sırasında sorgunun üzerinde çalıştırıldığı tablodur. Yani arama sırasında girilen sorgu, indeks tablosunda aranır.

Bu tabloya ters indeks (inverted index) denir. Dokümanlar, içerdikleri anahtar kelimelere göre indekslenir.

8.3 Basit Ters İndeks

Hoca daha önce gösterdiği basit ters indeksi tekrar gösterdi. Burada üç basit doküman vardır. Her anahtar kelime için, o anahtar kelimeyi içeren dokümanlar listelenir.

8.4 Gelişmiş Ters İndeks: Üçlüler

Ardından aynı oyuncak örneğin biraz daha gelişmiş biçimi gösterildi. İlk sütun yine anahtar kelimeler, ikinci sütun yine dokümanlardır. Ama bu kez her doküman bir üçlü (triple) olarak temsil edilir:

- **Birinci eleman:** Doküman kimliği (document ID).
- **İkinci eleman:** Bu anahtar kelimenin bu dokümanda kaç kez geçtiği.
- **Üçüncü eleman:** Anahtar kelimenin dokümandaki konumu (position).

Hocanın verdiği örnekler şunlardır:

- **applications:** 2 numaralı dokümanda yalnızca bir kez geçer. Konumu 3'tür, yani dokümanın başındaki ilk kelimeden başlayarak sayıldığında üçüncü kelimedir. Girdi $(2, 1, 3)$ biçimindedir.
- **mining:** Üç farklı dokümanda geçer: doküman 1, doküman 2 ve doküman 3. Her dokümanda yalnızca bir kez geçer. Konumu dokümana göre değişir: ilk iki dokümanda 2. konumda, üçüncü dokümanda 3. konumdadır. Girdiler $(1, 1, 2)$, $(2, 1, 2)$ ve $(3, 1, 3)$ biçimindedir.

  Ek bilgi (derste anlatılmadı): Slayttaki oyuncak örnek büyük olasılıkla Liu'nun kitabındaki şu üç dokümandır: d_1 "Web mining is useful.", d_2 "Usage mining applications.", d_3 "Web structure mining studies the Web hyperlink structure." Python ile kurulan indekste hocanın söylediği konumlar aynen çıkar: applications için $(2, 1, 3)$, mining için $(1, 1, 2)$, $(2, 1, 2)$, $(3, 1, 3)$. Bir kelime dokümanda birden çok kez geçerse, örneğin "Web" kelimesi d_3'te iki kez, konum alanı tek bir sayı değil konumların listesi olur: $(3, 2, [1, 6])$.

8.5 Gerçek Arama Motorlarındaki İndeks

Gerçekte global web arama motorlarının kullandığı indeks tabloları çok daha gelişmiştir. Üçlüler yerine daha fazla boyut kullanırlar. Yani her anahtar kelime için, bu kelimenin dokümanda nasıl geçtiğine dair daha fazla bilgi tutarlar:

- Anahtar kelime HTML'in meta etiketinde mi?
- Kalın (bold) yazılmış mı?
- İtalik (italic) yazılmış mı?

HTML dokümanlarında anahtar kelimelerin bulunduğu konum ve yazılış biçimi, sayfayı yazan kişinin bu kelimeye verdiği önemi ifade eder. Örneğin sayfa sahibi "applications" kelimesini başlıklardan birine (header) koymuşsa, bu kişi sayfasını tanımlarken bu anahtar kelimeye önem veriyordur. Bu bilgi aramada kullanılabilir. "applications" arayan birine sonuçlar gösterilirken, kelimenin başlıkta olduğu sayfaları göstermek bir avantaj olabilir. Global arama motorları ters indeks tablolarında burada gösterilenden çok daha fazla bilgi tutar.

9. Arama Sırasında Ters İndeksin Kullanılması

Kullanıcı sorgusunu arama motorunun metin kutusuna girdiğinde, arama motoru sorguyu alır ve ters indekse uygular, yani bir arama (lookup) yapar.

- **Tek kelimeli sorgu:** Örneğin “mining” aranıyorsa, motor ters indekste “mining” girdisini bulur ve bu girdiye karşılık gelen doküman listesini alır.
- **Çok kelimeli sorgu:** Örneğin “web mining” aranıyorsa, motor “web” girdisine gidip bazı dokümanları, “mining” girdisine gidip bazı dokümanları getirir. Sonra bu iki farklı girdiden gelen sonuçları birleştirir (merge).

Yani önce sorgu ters indeks tablosuna uygulanır, sonuçlar getirilir ve eşleştirilir. Sonuçlar kullanıcıya gösterilmeden önce yeniden sıralanır (re-rank). Bu sıralama iki tür yönteme dayanır:

- **İçerik tabanlı sıralama (content-based ranking):** TF-IDF skoruna dayanır.
- **Bağlantı tabanlı sıralama (link-based ranking):** PageRank ya da HITS gibi bağlantı tabanlı sıralama algoritmalarına dayanır.

TF-IDF yaklaşımı konuşuldu. Bağlantı tabanlı sıralama yaklaşımları, tarayıcı bölümünden sonra, yaklaşık üç hafta sonra işlenecek.

> **Ek bilgi (derste anlatılmadı):** Çok kelimeli sorguda birleştirme, Boolean sorgunun türüne göre değişir. AND sorgusunda iki posting listesinin kesişimi, OR sorgusunda birleşimi alınır. Posting listeleri doküman kimliğine göre sıralı tutulduğu için kesişim, iki listenin aynı anda bir kez taranmasıyla doğrusal sürede bulunur. Yukarıdaki üç dokümanlı örnekte büyük olasılıkla “web AND mining” için $\{d_1, d_3\}$, “web OR mining” için $\{d_1, d_2, d_3\}$ çıkar.

10. Arama Motorlarını Başarılı Kılan Etkenler

Farklı arama motorları farklı açılardan başarılıdır. Hoca bunun sebeplerini şöyle sıraladı:

- **Zengin indeks:** Başarılı motorlar indeks tablolarında diğer motorlara göre çok daha zengin bilgi tutar. Kelimenin başlıkta mı (title), gövdede mi (body) olduğu, vurgulanmış bir kelime olup olmadığı, kalın ya da italik yazılıp yazılmadığı gibi bilgiler.
- **Sorgu işleme (query processing):** Sorgunun nasıl genişletilebileceği konuşuldu. Motorlar bazen sorgu sınıflandırması (query classification) da yapar. Sorgular ön işlenerek ve ters indekste daha zengin bilgi tutularak aramanın kalitesi artırılır.
- **Sıralama algoritmaları:** En önemli etkenlerden biridir. PageRank, Google’ı başarılı kılan, iyi bilinen sıralama algoritmalarından biridir. Derste ileride başka arama sıralama algoritmaları da konuşulacak.

Dersin Toparlaması

Bu derste önce Boolean ve vektör uzayı modellerinin doküman ve sorgu temsili, TF-IDF ağırlıkları, kosinüs benzerliği ve ilgililik geri bildirimi tekrar edildi. Ardından indeksleyicinin dokümanları nasıl işlediği anlatıldı: Önce durak kelimeler atılır, sonra kök bulma uygulanır, sonra her terimin TF-IDF değeri hesaplanıp dokümanlar vektörle temsil edilir. Sonuçların kalitesi precision ($|A \cap B|/|B|$) ve recall ($|A \cap B|/|A|$) ile ölçülür. Web’de toplam ilgili doküman sayısı bilinmediği için recall hesaplanamaz ve Precision@10, @20, @30 kullanılır.

Son olarak ters indeksin yapısı ve arama sırasında nasıl kullanıldığı gösterildi. Her anahtar kelime için doküman listesi, gelişmiş biçimde de (doküman kimliği, geçme sayısı, konum) üçlüleri tutulur. Sorgudaki kelimeler indekste aranır, listeler birleştirilir ve sonuçlar TF-IDF ile bağlantı tabanlı yöntemlerle yeniden sıralanır. Bir sonraki konu web tarayıcısıdır (web crawling). Bağlantı tabanlı sıralama algoritmaları (PageRank, HITS) onun ardından işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.